STATISTICS WITH R PROGRAMMING

ASSIGNMENT – 3

CO3 – *A practical implementation using R data frames, data cleaning, reshaping, joins, reusable functions and performance analysis.*

MULTI-BRANCH CLINIC RECORD DATA-FRAME MANAGEMENT PIPELINE

Submitted by: K Sujith Sai Vighnesh
Register Number: 192311199
Course Code: ITA0412
Programme / Department: CSE
Institution : SIMATS
Academic Year: 2023-27

ABSTRACT

This assignment develops an integrated R programming pipeline for managing structured records in a multi-branch clinic environment. Five related datasets are used: Patients, Doctors, Appointments, LabTests and Billing. The data are synthetic and are used only to demonstrate data-frame operations and data-management concepts.

The program follows a complete processing sequence. It begins with requirements and schema design, creates data frames, inspects their structure, manipulates rows and columns, detects data-quality problems, reshapes monthly summaries, and joins related datasets through key fields. It then demonstrates file import and export, reusable functions, an integrated workflow, test cases, benchmarking and adaptation to changing input records.

The practical objective is not merely to produce a merged table, but to show how data integrity can be checked at every stage. Duplicate identifiers, missing values and unmatched references are identified and reported. Reshaping is checked by comparing totals before and after conversion. Exported data are read back to verify that the output is usable.

The supplied execution evidence shows RStudio completing the pipeline and saving generated files in the assignment3_output directory. The report documents the concepts, algorithms, code patterns, screenshots and interpretation of results.

Keywords

R programming, data frame, data cleaning, data validation, data reshaping, data merging, CSV, RDS, benchmarking, clinic records.

1. INTRODUCTION

A clinic operating across multiple branches maintains different categories of information. Patient registration data identify the people receiving services. Doctor data describe the staff and their departments. Appointment data record visits, dates, status and fees. Laboratory records describe tests associated with visits, while billing records capture payment information.

Although these datasets are related, they may be stored in separate files. A report that combines them must use reliable identifiers and consistent data types. If the patient identifier is misspelled or a duplicate patient row is retained, a join may create incorrect results. If a date is imported as ordinary text, date-based analysis becomes difficult. If missing values are silently removed, the final report may omit important appointments.

R is suitable for this kind of structured data processing. Its data-frame model supports mixed column types. Base R provides functions for selection, filtering, aggregation, sorting, joining and file operations. Packages such as reshape2 add convenient functions for converting data between wide and long layouts.

The assignment connects these operations into a modular pipeline. Each stage has a specific responsibility and produces output that can be checked before it is passed to the next stage.

2. PROBLEM STATEMENT, AIM AND SCOPE

2.1 Problem statement

The clinic datasets may contain duplicate patient identifiers, missing names, inconsistent text, non-standard dates and references to records that do not exist in the related master table. Separate data files also make it difficult to produce a complete view of appointments, laboratory tests and billing. A structured process is therefore required to validate, clean, transform, combine and export the data without losing track of unresolved records.

2.2 Aim

To design, develop and execute an integrated R data-frame management pipeline for a multi-branch clinic records analytics system using data-frame creation, manipulation, cleaning, reshaping, merging, file import/export, reusable functions, testing and performance benchmarking.

2.3 Scope

- Create and process synthetic clinic datasets.
- Demonstrate data-frame operations and validation rules.
- Produce a combined appointment-centred master dataset.
- Export processed datasets and reports to files.
- Measure the time taken by selected operations on different dataset sizes.

2.4 Out of scope

The program is an academic R demonstration. It is not a deployed electronic health record system and does not implement authentication, encryption, live database connectivity, clinical decision support or real patient workflows.

3. OBJECTIVES AND EXPECTED OUTCOMES

1. Design the entities, attributes, primary keys, foreign keys and relationships.
2. Create the five synthetic data frames and inspect their structure and summary.
3. Select, filter, sort, update, append and remove data-frame rows and columns.
4. Detect missing data, duplicate keys, inconsistent strings and invalid references.
5. Convert monthly summary data between wide and long formats.
6. Join patient, doctor, appointment, lab and billing records.
7. Import CSV, TSV and delimited text files with suitable data types.
8. Export versioned CSV and RDS files and verify the read-back results.
9. Implement reusable functions for cleaning, reshaping, merging and exporting.
10. Integrate the modules and execute acceptance tests.
11. Benchmark merge, summary and file operations on different data sizes.
12. Generate documentation and a final summary of the pipeline.

Expected outcomes

At completion, the student should be able to explain the role of data frames, keys, joins, cleaning rules, reshape operations, reusable functions and test cases. The student should also be able to interpret the performance graph and identify why data validation is essential before producing a combined report.

4. SOFTWARE REQUIREMENTS AND EXECUTION ENVIRONMENT

Item	Details
Operating system	Windows 11 x64, as displayed in the R session
R version	R version 4.4.2 – Pile of Leaves
IDE	RStudio
Package	reshape2 1.4.5, shown in the execution session
Input formats	CSV, TSV and delimited text
Output formats	CSV, RDS and PNG graph
Editor/viewer	RStudio, spreadsheet application and text editor
Execution method	Run the complete script using source() in RStudio Console

Execution procedure

13. Download the supplied Assignment 3 .R script.
14. Open RStudio and set the working directory to the folder containing the script.
15. Run source(file.choose()) in the Console.
16. Select the Assignment 3 script in the file selection dialog.
17. Wait until the final completion message appears.
18. Open the assignment3_output directory to inspect generated files.

```
setwd("C:/Users/Dell/Downloads")

source(file.choose())
```

The file.choose() method is useful when the script is saved in a different folder or when the exact file path is not known.

5. SYSTEM ARCHITECTURE AND WORKFLOW

The pipeline is arranged in stages so that each operation can be inspected independently. The first stage prepares source data. The next stages validate the schema, clean values, reshape monthly summaries and merge related entities. The final stages test the output, benchmark operations and export the results.

SOURCE DATA CREATION / FILE IMPORT

↓

SCHEMA AND KEY VALIDATION

↓

DATA CLEANING AND STANDARDISATION

↓

DATA-FRAME MANIPULATION AND RESHAPING

↓

MULTI-STAGE MERGING

↓

REUSABLE FUNCTIONS AND INTEGRATED PIPELINE

↓

TESTING AND PERFORMANCE BENCHMARKING

↓

VERSIONED EXPORTS AND FINAL REPORT

5.1 Why a staged design is useful

- Errors can be isolated to a particular processing stage.
- Intermediate row counts can be checked before and after transformation.
- Reusable functions reduce repeated code.
- The same pipeline can be rerun with new input files.
- The final report can include diagnostic counts and test results.

6. DATA MODEL AND DATA DICTIONARY

Dataset	Key fields	Purpose
Patients	patient_id, patient_name, age, gender, branch_id, registration_date	Patient registration details
Doctors	doctor_id, doctor_name, department, branch_id	Doctor and department information
Appointments	appointment_id, patient_id, doctor_id, branch_id, appointment_date, status, fee	Visit schedule and appointment details
LabTests	lab_id, appointment_id, test_name, result_value, result_status	Diagnostic test records
Billing	bill_id, appointment_id, amount, payment_status, payment_date	Invoice and payment records

6.1 Relationships

Parent	Child	Relationship	Link field
Patients	Appointments	One-to-many	patient_id
Doctors	Appointments	One-to-many	doctor_id
Appointments	LabTests	One-to-many	appointment_id
Appointments	Billing	One-to-one in sample	appointment_id

A primary key uniquely identifies one row in its own table. A foreign key refers to a key in another table. These keys make it possible to combine records and to detect references that cannot be matched.

TASK 1 – REQUIREMENTS AND BLUEPRINT

Purpose: The first task defines what information the pipeline must manage and how the information is connected. Planning the entities and keys before coding prevents later join errors. The requirements blueprint lists each dataset, its role, key fields, important relationships and validation conditions.

Concepts and R functions

R can represent a blueprint as a data frame. Character vectors store the names of entities and their key fields. print() displays the table, while names() and str() can be used to inspect its columns.

Expected output and interpretation

Expected result: five datasets are described. Patients and Doctors connect to Appointments. LabTests and Billing connect to Appointments. The acceptance criteria require unique identifiers, valid references, retained appointment rows, preserved monthly totals and reproducible exports.

Algorithm / procedure

19. List the entities required by the clinic scenario.
20. Select a primary key for each entity.
21. Identify foreign keys and relationship cardinalities.
22. Define the checks that must pass before reporting.
23. Document the intended outputs.

Illustrative R code

```
blueprint <- data.frame(entity, primary_key, foreign_key)
print(blueprint)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 2 – CREATE AND VALIDATE DATA FRAMES

Purpose: This task creates the synthetic datasets used throughout the assignment. A data frame is a two-dimensional R object in which columns can contain different data types. The script creates patient, doctor, appointment, laboratory and billing records.

Concepts and R functions

data.frame() constructs the tables. as.Date() represents date values. factor() can represent categories. str() displays the internal structure; summary() gives descriptive summaries; dim() returns the row and column counts; names() returns column names; class() identifies object classes.

Expected output and interpretation

Expected result: the initial patient data includes an intentionally repeated identifier and a missing field so that validation can be demonstrated. The appointment data contains 20 records. The script inspects the structure before attempting to clean or join the records.

Algorithm / procedure

24. Create each data frame with named columns.
25. Use consistent identifiers for related rows.
26. Inspect the dimensions and column names.
27. Check the class of date and categorical fields.
28. Create controlled test defects for later cleaning demonstrations.

Illustrative R code

```
str(patients)
summary(patients)
dim(appointments)
names(doctors)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 3 – DATA-FRAME MANIPULATION

Purpose: This task demonstrates the operations used to retrieve and modify selected information. Data-frame manipulation is central to exploratory analysis because it allows the programmer to focus on relevant records without changing the original source unnecessarily.

Concepts and R functions

The $ operator accesses a named column. [ ] selects rows or columns. [[ ]] extracts a single column. subset() filters rows. order() sorts rows. rbind() appends rows and cbind() combines columns. table(), aggregate() and tapply() summarise data by categories.

Expected output and interpretation

Expected result: the script selects patient fields, filters patients and appointments, creates derived fields, changes a value, appends and removes sample rows, sorts by fee and calculates grouped appointment counts and fee summaries.

Algorithm / procedure

29. Select a single column using $.
30. Select multiple columns using [ ].
31. Filter records with logical conditions.
32. Create a derived column such as an age group.
33. Sort records using order().
34. Summarise appointments by doctor and status.

Illustrative R code

```
subset(appointments, status == "Complete")
aggregate(fee ~ doctor_id, appointments, mean)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 4 – DATA CLEANING AND STANDARDISATION

Purpose: Real-world datasets commonly contain incomplete or inconsistent values. Cleaning makes records more consistent while preserving a trace of unresolved problems. The demonstration includes duplicate identifiers, missing names, whitespace, inconsistent case, date formats and unmatched foreign keys.

Concepts and R functions

is.na() identifies missing values. trimws() removes leading and trailing whitespace. tolower() and toupper() standardise letter case. duplicated() detects repeated values. complete.cases() identifies complete rows. as.Date() converts character dates into Date values.

Expected output and interpretation

Expected result: duplicate patient identifiers are removed according to the program's rule. The missing patient name is flagged for manual review. Appointment references that do not match a patient record remain identifiable for follow-up rather than being silently treated as valid.

Algorithm / procedure

35. Inspect missing values and duplicates.
36. Standardise identifiers and text fields.
37. Convert date strings to Date objects.
38. Check foreign-key references against parent tables.
39. Flag records that need manual review.
40. Record row counts before and after cleaning.

Illustrative R code

```
patients$patient_id <- trimws(patients$patient_id)
patients <- patients[!duplicated(patients$patient_id), ]
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 5 – MELTING AND CASTING

Purpose: Monthly clinic totals are often stored in wide format, with one column for each month. Long format stores the month in one column and the corresponding value in another. Long format is convenient for grouped analysis and plotting, while wide format is often easier to read in a compact report.

Concepts and R functions

reshape2::melt() converts wide data to long format by identifying the key columns and gathering the remaining columns.

reshape2::dcast() converts long data back to wide format using a formula that defines rows and columns.

Expected output and interpretation

Expected result: the branch-month summary is melted into branch_id, month and appointments columns. It is then cast back into a wide table. The total is compared before and after reshaping; the sample total is 1005.

Algorithm / procedure

41. Create a wide monthly summary.
42. Apply melt() with branch_id as the identifier.
43. Inspect the resulting long-format rows.
44. Apply dcast() to reconstruct the wide table.
45. Compare totals before and after reshaping.
46. Explain why both layouts are useful.

Illustrative R code

```
long_data <- reshape2::melt(monthly_wide, id.vars = "branch_id")
wide_data <- reshape2::dcast(long_data, branch_id ~ variable)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 6 – MULTI-STAGE MERGING

Purpose: This task creates an appointment-centred master dataset by joining the related clinic tables. Merging is necessary because each source table contains only part of the overall information.

Concepts and R functions

merge() combines data frames using matching key columns. An inner join keeps matching records only. A left join retains all records from the left table. A full join retains keys from both tables. all.x=TRUE performs a left join and all=TRUE performs a full join.

Expected output and interpretation

Expected result: appointments are joined to patients and doctors, then laboratory and billing information is attached using appointment_id. The program compares join types and reports unmatched references. A left join is appropriate when all appointment records must remain visible.

Algorithm / procedure

47. Check that join keys exist in both datasets.
48. Ensure key columns use compatible data types.
49. Join appointments to patients.
50. Join the result to doctors.
51. Attach laboratory and billing records.
52. Compare inner, left and full join results.
53. Inspect unmatched records and missing related data.

Illustrative R code

```
master <- merge(appointments, patients, by = "patient_id", all.x = TRUE)
master <- merge(master, doctors, by = "doctor_id", all.x = TRUE)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 7 – FILE IMPORT AND EXPORT

Purpose: This task demonstrates how data can be exchanged between R and external files. Import settings matter because a wrong separator or data type can cause columns to be read incorrectly. Export verification confirms that the output can be read again.

Concepts and R functions

read.csv() reads comma-separated files. read.delim() reads tab-separated files. read.table() supports custom separators. readLines() inspects raw text. write.csv() writes a CSV file. saveRDS() and readRDS() preserve and restore an R object.

Expected output and interpretation

Expected result: source CSV, TSV and delimited text files are imported. Cleaned and merged datasets are exported with timestamped filenames. The RDS output is read back and compared with the original object to verify that the structure is preserved.

Algorithm / procedure

54. Create or locate source files.
55. Inspect raw lines and separators.
56. Import files with appropriate functions.
57. Check the dimensions and classes after import.
58. Export cleaned and merged data to CSV.
59. Save the master object in RDS format.
60. Read files back and verify row counts and object equality.

Illustrative R code

```
patients <- read.csv("patients.csv")
write.csv(patients, "patients_clean.csv", row.names = FALSE)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 8 – REUSABLE FUNCTION LIBRARY

Purpose: Reusable functions separate the program into smaller units. A function accepts inputs, performs a task and returns a result. This makes the pipeline easier to test, reuse and maintain.

Concepts and R functions

function() defines a function. Arguments provide input values. return() can explicitly return a result. stop() terminates execution when an input is invalid. warning() reports a possible issue. A list can return multiple objects such as cleaned data and validation counts.

Expected output and interpretation

Expected result: the script defines helpers for validation and cleaning, monthly reshaping, clinic merging and versioned export. It also tests invalid or empty inputs to check that the functions respond safely.

Algorithm / procedure

61. Define a function for key validation and cleaning.
62. Return both accepted and rejected records where appropriate.
63. Define a helper for monthly reshaping.
64. Define a helper for multi-stage merging.
65. Define a versioned export helper.
66. Test normal, invalid and empty inputs.

Illustrative R code

```
clean_data <- function(x) {
 x$patient_id <- trimws(x$patient_id)
 x[!duplicated(x$patient_id), ]
}
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 9 – INTEGRATED PIPELINE AND TESTING

Purpose: The integrated pipeline connects the individual modules into one execution sequence. Testing checks whether the program behaves according to the acceptance criteria and whether important data properties are preserved.

Concepts and R functions

An integrated function calls the cleaning, reshaping, merging and export steps in a controlled order. A test compares an actual result with an expected condition. A fixed random seed supports repeatable synthetic data generation.

Expected output and interpretation

Expected result: the pipeline processes the appointment data and records diagnostic counts. The supplied execution report indicates that the integrated pipeline completed and reports six acceptance checks. The checks include key uniqueness, appointment retention, unmatched references, reshape-total preservation, RDS read-back and reproducibility.

Algorithm / procedure

67. Run the cleaning functions.
68. Run the join sequence.
69. Collect diagnostic counts.
70. Execute each acceptance test.
71. Check that monthly totals are preserved.
72. Verify that exported RDS data can be read back.
73. Repeat the workflow to assess reproducibility.

Illustrative R code

```
result <- run_clinic_pipeline(...)
stopifnot(nrow(result$appointments) == expected_rows)
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 10 – PERFORMANCE AND MAINTAINABILITY

Purpose: The benchmark measures how selected operations behave when the number of records increases. It compares merge, grouped summary and CSV round-trip operations for 5,000 and 500,000 records.

Concepts and R functions

system.time() records elapsed execution time. object.size() estimates the memory size of an R object. A benchmark should compare equivalent operations and should also check correctness. The same code can run at different input sizes to observe scaling behaviour.

Expected output and interpretation

Observed performance values shown in the earlier execution log are machine-dependent. The supplied graph visualises elapsed time by record count and operation. The benchmark also checks that the merge result retains the expected number of rows.

Algorithm / procedure

74. Generate synthetic datasets at both record counts.
75. Measure merge execution time.
76. Measure grouped summary execution time.
77. Measure CSV write/read round-trip time.
78. Record the output row count and object size.
79. Plot the timing results.
80. Interpret the differences without treating them as universal timings.

Illustrative R code

```
timing <- system.time(merge(x, y, by = "id"))
print(timing["elapsed"])
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 11 – CHANGED REQUIREMENT AND LARGE-SCALE ADAPTATION

Purpose: This task demonstrates how the workflow can respond when records arrive in separate batches or the clinic adds new operational data. A later file may contain a correction to a record already received.

Concepts and R functions

The batch workflow combines data from different source days and uses a precedence rule to select the latest record for a repeated appointment identifier. Sorting and duplicated() can support this demonstration. In a production environment, the source timestamp and audit history should be retained.

Expected output and interpretation

Expected result: the program demonstrates a multi-batch update and expansion of doctor information. It illustrates the need to decide whether a corrected record replaces an earlier value or whether both versions should be retained for audit purposes.

Algorithm / procedure

81. Create two batches of appointment records.
82. Introduce a correction in the later batch.
83. Combine the batches and retain source information.
84. Sort according to the source precedence rule.
85. Select the latest row for each appointment key.
86. Add new doctor records and update the master data.
87. Document the audit-history limitation.

Illustrative R code

```
combined <- rbind(day1, day2)
combined <- combined[order(combined$source_day), ]
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

TASK 12 – DOCUMENTATION AND FINAL SUMMARY

Purpose: The final task prepares supporting documentation for the data model and the completed execution. A data dictionary describes fields and their meaning. A summary report records counts, checks, benchmarks and generated file paths.

Concepts and R functions

sessionInfo() records the R version, operating system and loaded packages. A summary function can combine results into a named list. write.csv() can export the data dictionary, while saveRDS() stores the final summary as an R object.

Expected output and interpretation

Expected result: the pipeline generates a data dictionary and summary report, prints the execution environment and displays a final completion message. The generated files are saved under assignment3_output.

Algorithm / procedure

88. Document the datasets and important fields.
89. Summarise the row counts and validation results.
90. Record benchmark results and generated file paths.
91. Save the final summary report.
92. Print the R session information.
93. Confirm the final completion message.

Illustrative R code

```
sessionInfo()
saveRDS(summary_report, "final_summary_report.rds")
```

Note: The complete executable implementation is in the accompanying R script. The code extract on this page illustrates the principal operation for the task.

18. INTEGRATED VALIDATION AND TEST CASES

Testing is performed to confirm that the pipeline meets its data-integrity requirements. A successful run should not be judged only by the absence of an error message. The output must also satisfy expected conditions such as key uniqueness, retained appointment rows and consistent totals.

Test	What is checked	Reason
Unique patient key	No duplicate patient_id in cleaned patient data	Prevents ambiguous patient matching
Appointment retention	All expected appointment records remain in the appointment-centred output	Prevents accidental record loss
Invalid patient references	Unmatched patient IDs are identified	Makes referential-integrity issues visible
Reshape total	Total before and after melt/cast is equal	Confirms values are preserved
RDS read-back	Restored object matches saved object	Checks export integrity
Reproducibility	Repeated run produces expected stable output	Supports repeatable analysis

Interpreting test results

A TRUE result indicates that the condition being checked was satisfied. A FALSE result requires investigation. A data-quality warning does not always mean the whole pipeline failed: an unmatched patient reference may be an expected defect deliberately inserted for the assignment, provided it is correctly detected and reported.

The supplied execution summary indicates six systematic checks passed. The pipeline also completed and generated the output directory.

19. PERFORMANCE BENCHMARKING

The benchmark evaluates three operations: merging related records, calculating grouped summaries and writing then reading a CSV file. The operations are measured at two input sizes, 5,000 and 500,000 records. This provides a basic indication of how execution time changes as the data volume increases.

Record count	Merge (sec)	Summary (sec)	CSV round-trip (sec)
5,000	0.03	0.00	0.06
500,000	3.45	0.18	3.16

These values are taken from the earlier execution output and should be treated as observations from that particular computer and run. Execution times can change with processor speed, available memory, storage performance, R version and other applications running at the same time.

Analysis

- Merge and file round-trip time increase substantially at the larger data size.
- The grouped summary operation is comparatively faster in the recorded run.
- The benchmark checks row counts so that speed is considered together with correctness.
- The graph is useful for comparing relative elapsed times, but it is not a universal performance guarantee.

20. RSTUDIO EXECUTION SCREENSHOT

The screenshot below records the final RStudio execution state. The Console displays the final pipeline completion message and the output directory. The Environment pane shows the functions and objects created during execution.

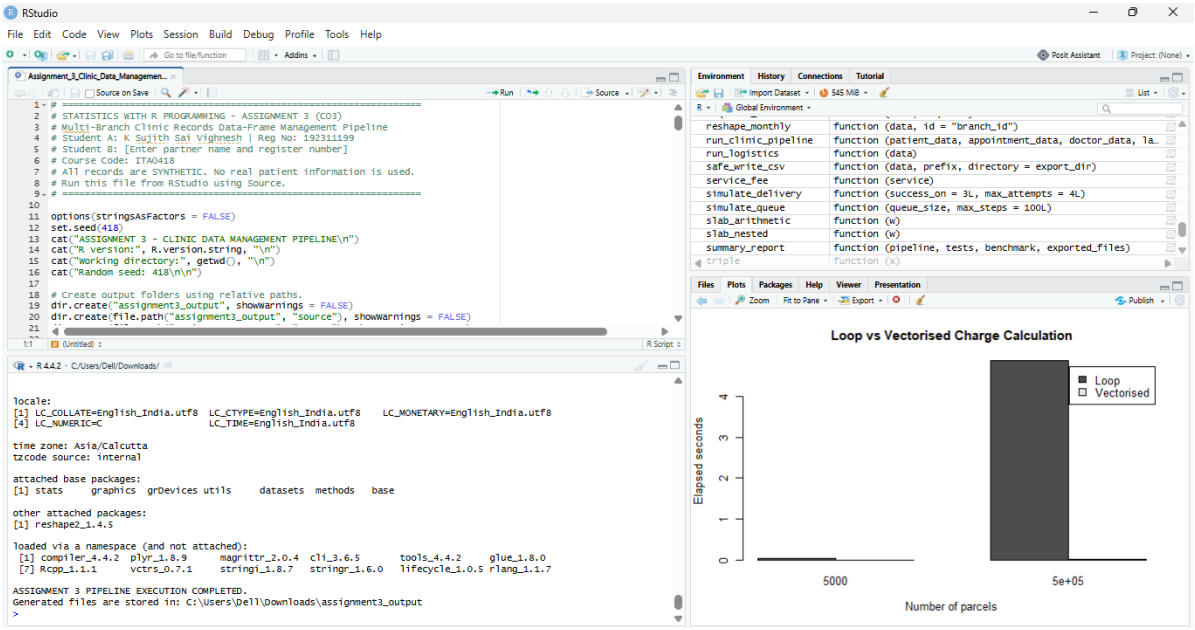


Figure 1. RStudio execution evidence: script, Environment, Plots and Console completion message.

The final message shown in the Console is: ASSIGNMENT 3 PIPELINE EXECUTION COMPLETED. Generated files are stored in the assignment3_output directory.

21. PERFORMANCE GRAPH OUTPUT

Task 10 generates a bar chart titled “Clinic Pipeline Performance Comparison”. The x-axis identifies the appointment-record size and the y-axis shows elapsed seconds. Separate bars compare merge, summary and CSV round-trip operations.

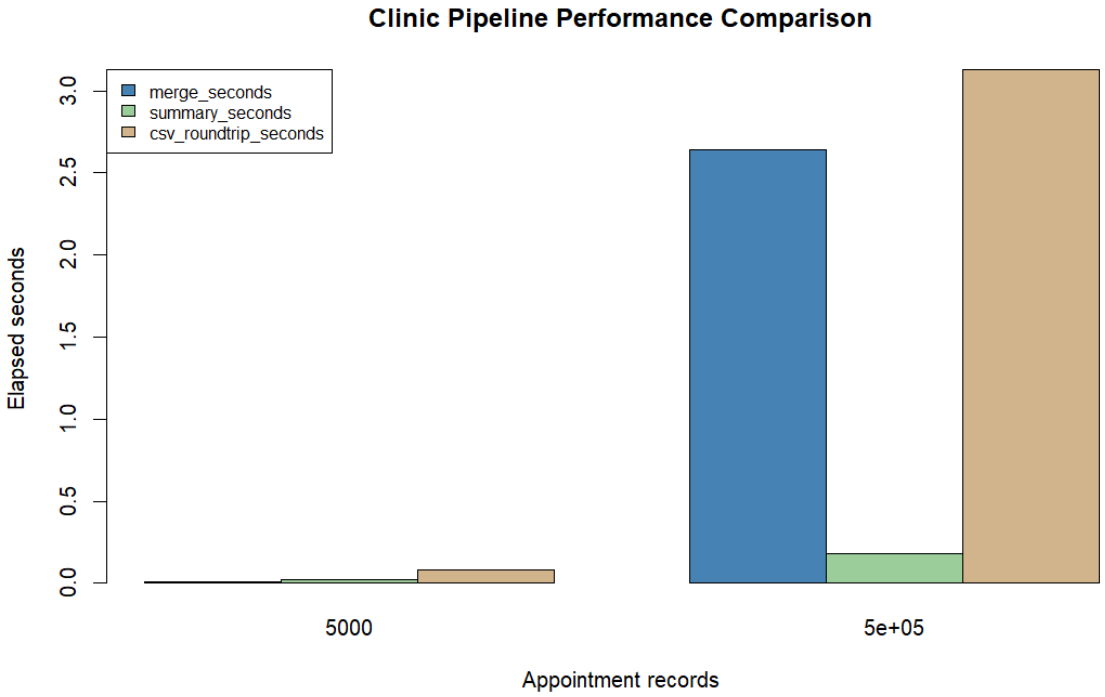


Figure 2. Clinic Pipeline Performance Comparison generated by the R script.

The graph shows that the larger dataset requires more time for merging and CSV round-trip operations. The summary calculation takes less time in comparison. This visual comparison supports the discussion of scalability and execution efficiency.

22. DATA DICTIONARY OUTPUT

A data dictionary provides a compact description of the generated datasets, key fields, important source columns and validation rules. It helps a reviewer understand the meaning of the exported tables without reading the complete R script.

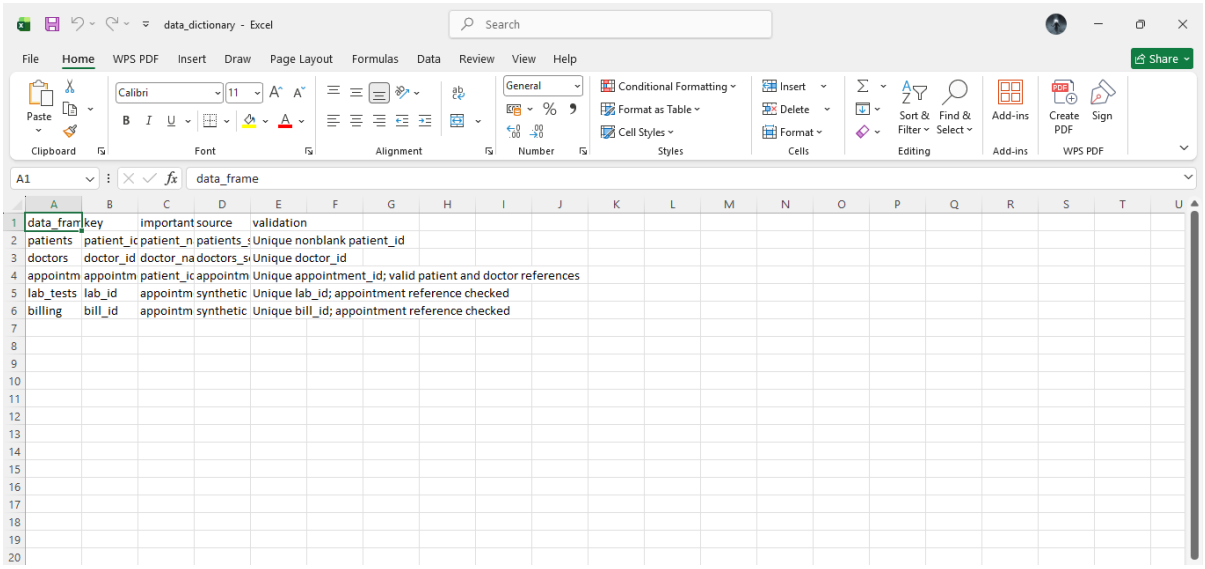


Figure 3. Data dictionary exported by the pipeline and opened in a spreadsheet application.

The displayed dictionary identifies the datasets and their key fields. Validation descriptions explain rules such as unique patient IDs and valid references from appointments, lab tests and billing records.

23. GENERATED FILES AND EXPORT DIRECTORY

The program creates a dedicated output directory so that generated files are kept separate from the original script. The output structure includes source files, exported processed data and report artifacts.

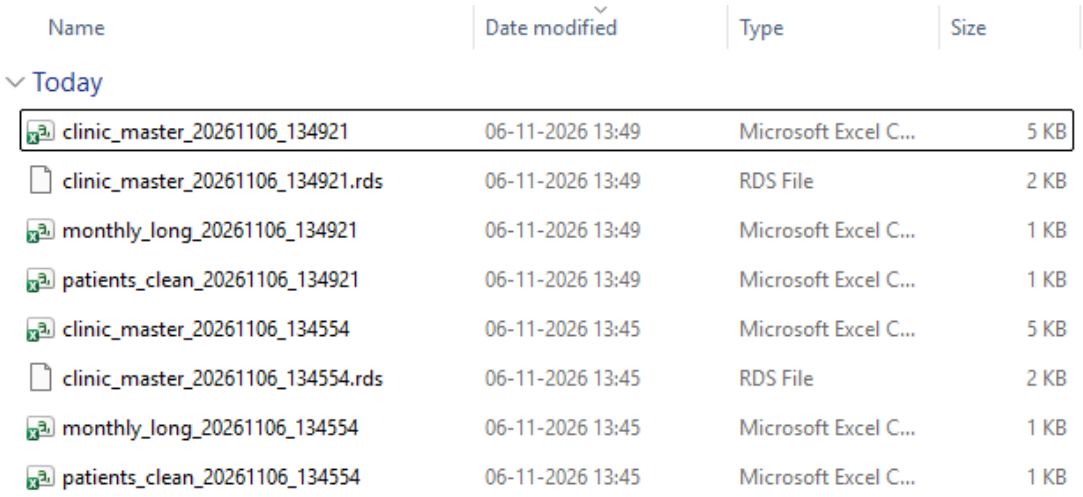
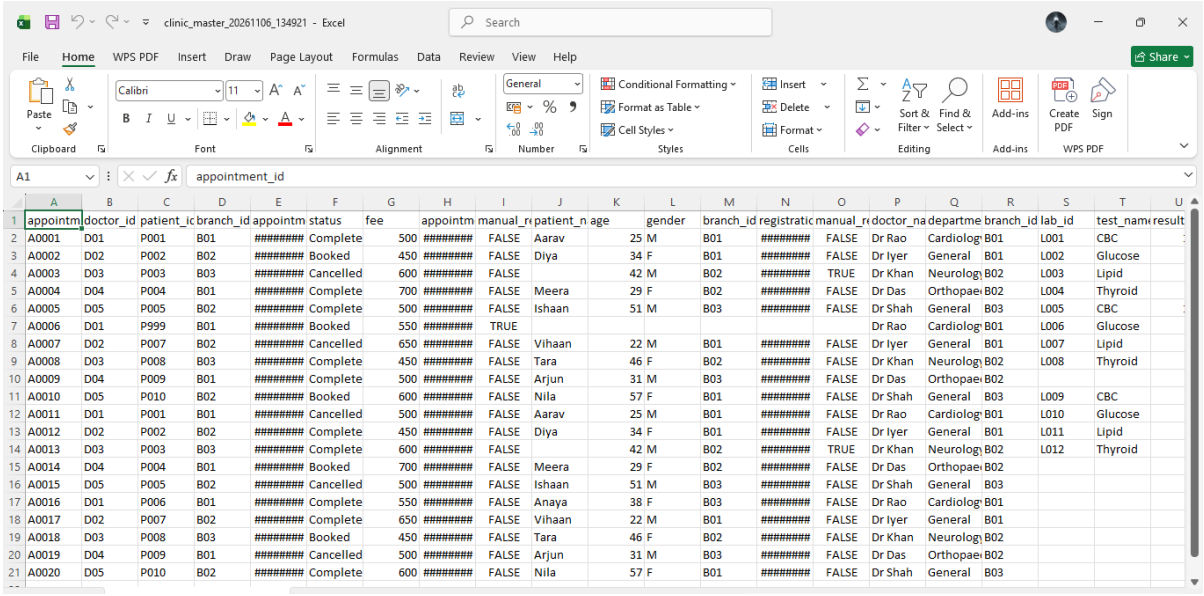


Figure 4. Timestamped generated files in the Assignment 3 output directory.

Timestamped filenames make it easier to distinguish output from different runs. CSV files can be inspected in spreadsheet software, while RDS files preserve R-specific object structure for later R sessions.

24. CLINIC MASTER DATASET OUTPUT

The clinic master dataset combines appointment information with related patient, doctor, laboratory and billing fields. It provides a single view that can support branch-level reporting and further analysis.



appointm	doctor_id	patient_id	branch_id	appointm	status	fee	appointm	manual_r	patient_n	age	gender	branch_id	registratic	manual_r	doctor_na	departme	branch_id	lab_id	test_nam	result
A0001	D01	P001	B01	########	Complete	500	########	FALSE	Aarav	25	M	B01	########	FALSE	Dr Rao	Cardiolog	B01	L001	CBC	
A0002	D02	P002	B02	########	Booked	450	########	FALSE	Diya	34	F	B01	########	FALSE	Dr Iyer	General	B01	L002	Glucose	
A0003	D03	P003	B03	########	Cancelled	600	########	FALSE		42	M	B02	########	TRUE	Dr Khan	Neurolog	B02	L003	Lipid	
A0004	D04	P004	B01	########	Complete	700	########	FALSE	Meera	29	F	B02	########	FALSE	Dr Das	Orthopae	B02	L004	Thyroid	
A0005	D05	P005	B02	########	Complete	500	########	FALSE	Ishaan	51	M	B03	########	FALSE	Dr Shah	General	B03	L005	CBC	
A0006	D01	P999	B01	########	Booked	550	########	TRUE							Dr Rao	Cardiolog	B01	L006	Glucose	
A0007	D02	P007	B02	########	Cancelled	650	########	FALSE	Vihaan	22	M	B01	########	FALSE	Dr Iyer	General	B01	L007	Lipid	
A0008	D03	P008	B03	########	Complete	450	########	FALSE	Tara	46	F	B02	########	FALSE	Dr Khan	Neurolog	B02	L008	Thyroid	
A0009	D04	P009	B01	########	Complete	500	########	FALSE	Arjun	31	M	B03	########	FALSE	Dr Das	Orthopae	B02			
A0010	D05	P010	B02	########	Booked	600	########	FALSE	Nila	57	F	B01	########	FALSE	Dr Shah	General	B03	L009	CBC	
A0011	D01	P001	B01	########	Cancelled	500	########	FALSE	Aarav	25	M	B01	########	FALSE	Dr Rao	Cardiolog	B01	L010	Glucose	
A0012	D02	P002	B02	########	Complete	450	########	FALSE	Diya	34	F	B01	########	FALSE	Dr Iyer	General	B01	L011	Lipid	
A0013	D03	P003	B03	########	Complete	600	########	FALSE		42	M	B02	########	TRUE	Dr Khan	Neurolog	B02	L012	Thyroid	
A0014	D04	P004	B01	########	Booked	700	########	FALSE	Meera	29	F	B02	########	FALSE	Dr Das	Orthopae	B02			
A0015	D05	P005	B02	########	Cancelled	500	########	FALSE	Ishaan	51	M	B03	########	FALSE	Dr Shah	General	B03			
A0016	D01	P006	B01	########	Complete	550	########	FALSE	Anaya	38	F	B03	########	FALSE	Dr Rao	Cardiolog	B01			
A0017	D02	P007	B02	########	Complete	650	########	FALSE	Vihaan	22	M	B01	########	FALSE	Dr Iyer	General	B01			
A0018	D03	P008	B03	########	Booked	450	########	FALSE	Tara	46	F	B02	########	FALSE	Dr Khan	Neurolog	B02			
A0019	D04	P009	B01	########	Cancelled	500	########	FALSE	Arjun	31	M	B03	########	FALSE	Dr Das	Orthopae	B02			
A0020	D05	P010	B02	########	Complete	600	########	FALSE	Nila	57	F	B01	########	FALSE	Dr Shah	General	B03			

Figure 5. Clinic master dataset exported from the integrated pipeline.

The spreadsheet preview shows appointment identifiers, doctor and patient identifiers, branch and appointment information, along with associated fields from the joined datasets. Date columns may display as hash symbols in a spreadsheet when the column width is too narrow; widening the column reveals the stored date value.

25. SOURCE AND CLEANED DATASET EVIDENCE

The following screenshots show the source and processed tables created by the pipeline. They provide evidence that the exported data can be opened outside RStudio.

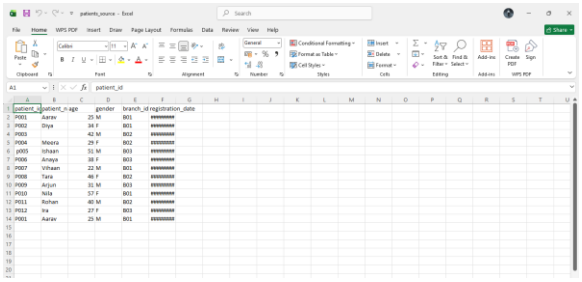


Figure 6. Patient source dataset

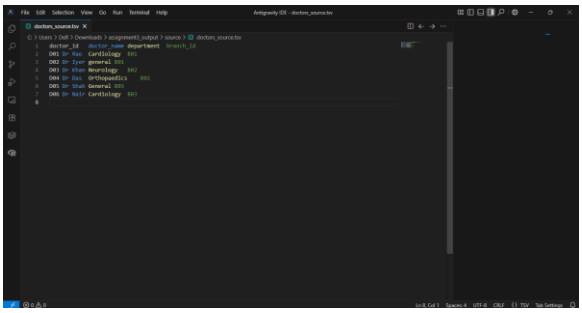


Figure 7. Doctors source TSV

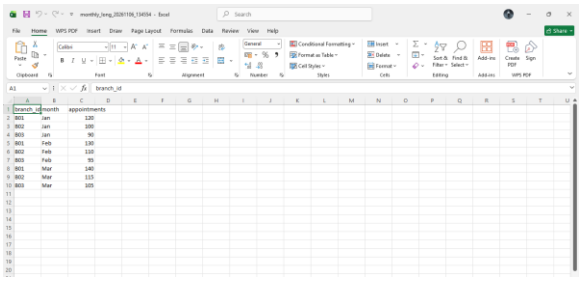


Figure 8. Monthly long-format dataset

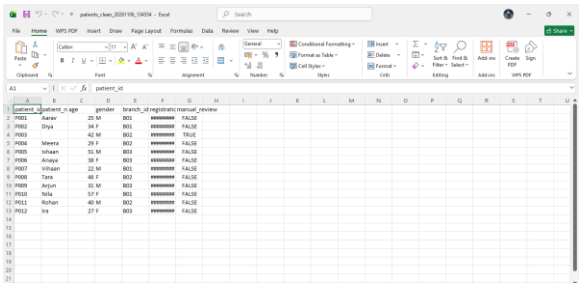


Figure 9. Cleaned patient dataset

26. RESULTS, DISCUSSION AND CONCLUSION

26.1 Result

The Assignment 3 pipeline was executed successfully in RStudio. The final Console message confirms completion and identifies the assignment3_output folder. The program demonstrates all twelve stages, including data-frame creation, manipulation, cleaning, reshaping, merging, import/export, reusable functions, integration testing, performance measurement, changed-requirement handling and documentation.

26.2 Discussion

The practical work demonstrates that data preparation is an essential part of data analysis. A join can execute without a syntax error and still produce an incomplete or misleading result if keys are invalid. For this reason, the pipeline checks primary keys, foreign keys, row counts and totals. The use of a left join for the appointment-centred output helps retain appointment rows even when a related patient record is missing.

26.3 Limitations

- All clinic records are synthetic demonstration data.
- The sample billing relationship is simplified for the assignment.
- The benchmark timings depend on the execution environment.
- The batch precedence rule is a teaching example and not a full audit-history system.
- The program is not a production clinical information system.

26.4 Conclusion

This assignment provided practical experience in building a structured data-management workflow in R. The pipeline demonstrates how related data frames can be validated, cleaned, reshaped, merged, exported and tested. Reusable functions improve maintainability, while benchmarking illustrates the effect of increasing data volume. The generated reports and screenshots provide evidence of successful execution and output inspection.

27. VIVA QUESTIONS AND EXECUTION REFERENCE

Question	Answer
What is a data frame?	A two-dimensional tabular R object with rows and named columns.
What is a primary key?	A field that uniquely identifies each record in a table.
What is a foreign key?	A field that refers to a key in another related table.
What is a left join?	A join that retains all rows from the left data frame and adds matching right-side values.
Why is data cleaning needed?	To identify missing, duplicated, inconsistent or invalid values before analysis.
What is melting?	Converting wide data into long format.
What is casting?	Converting long data into a wide layout, optionally aggregating values.
Why use RDS?	It preserves R object structure and data types more faithfully than CSV.
What does system.time() measure?	The time taken to evaluate an R expression, including elapsed time.
Why verify exported files?	To ensure the output can be read back and retains the expected records.

Execution command

```
setwd("C:/Users/Dell/Downloads")
source(file.choose())
```

Select Assignment_3_Clinic_Data_Management_Pipeline.R when the file chooser opens. Wait for the completion message and then inspect the assignment3_output folder.

Submission note: submit this report together with the executable R script and the generated output files if required by the course instructor.